

AWS ELB

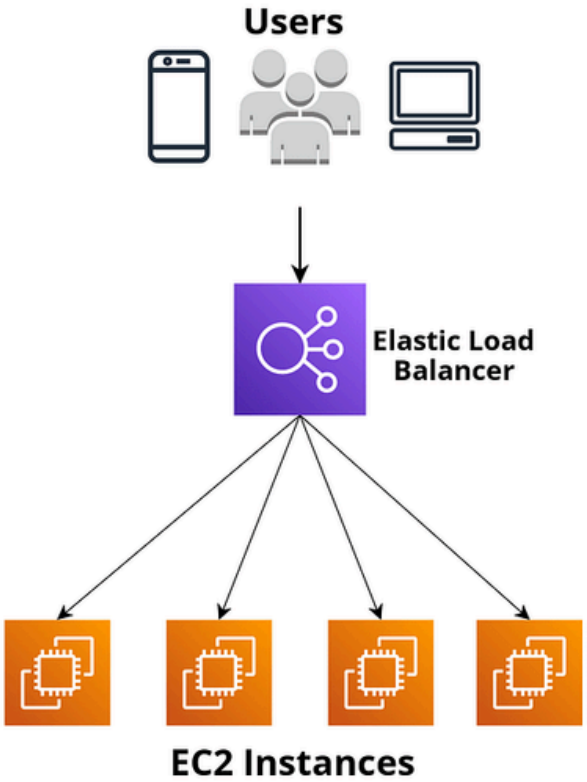
Elastic Load Balancer & Target Group (TG)

Concept Overview:

Introduction to ELB	1
Types of ELB	2
Why & When use ELB	3
Target Group(TG)	4
Application Load Balancer(ALB)	5
Hands-on practice	6

About ELB:

Elastic Load Balancing automatically distributes your incoming traffic across multiple targets, such as EC2 instances, containers, and IP addresses, in one or more Availability Zones. It monitors the health of its registered targets, and routes traffic only to the healthy targets. Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.



Types of ELB:

There are Four types of Load Balancer.

Application Load Balancer
Routes and load balances at the application layer (HTTP/HTTPS), and supports path-based routing. An Application Load Balancer can route requests to ports on one or more registered targets, such as EC2 instances, in your virtual private cloud (VPC).

Network Load Balancer
Routes and load balances at the transport layer (TCP/UDP Layer-4), based on address information extracted from the Layer-4 header. Network Load Balancers can handle traffic bursts, retain the source IP of the client, and use a fixed IP for the life of the load balancer.

Gateway Load Balancer
Distributes traffic to a fleet of appliance instances. Provides scale, availability, and simplicity for third-party virtual appliances, such as firewalls, intrusion detection and prevention systems, and other appliances. Gateway Load Balancers work with virtual appliances that support the GENEVE protocol. Additional technical integration is required, so make sure to consult the user guide before choosing a Gateway Load Balancer.

Classic Load Balancer
Routes and load balances either at the transport layer (TCP/SSL), or at the application layer (HTTP/HTTPS).

Why & When use ELB:

Why ELB:
A load balancer distributes workloads across multiple compute resources, such as virtual servers. Using a load balancer increases the availability and fault tolerance of your applications.

You can add and remove compute resources from your load balancer as your needs change, without disrupting the overall flow of requests to your applications.

You can configure health checks, which monitor the health of the compute resources, so that the load balancer sends requests only to the healthy ones. You can also offload the work of encryption and decryption to your load balancer so that your compute resources can focus on their main work.

Why & When use ELB:

When to use ELB:

- **Distributing traffic across multiple instances:** Use ELB to automatically balance incoming requests among multiple EC2 instances for better performance.

- **High availability and fault tolerance**: Ideal when you need your application to stay online even if one or more instances fail.

- **Auto Scaling integration:** When using Auto Scaling groups, ELB ensures traffic is routed to newly launched healthy instances.

- **SSL/TLS termination:** Use ELB to manage and offload HTTPS encryption, simplifying certificate management.

- **Multi-AZ deployments:** Best when you want to route traffic to healthy instances across multiple Availability Zones for disaster recovery.

Target Group(TG):

Target Group (TG) is a component of Elastic Load Balancing that routes traffic to one or more registered targets, such as EC2 instances, IP addresses, or Lambda functions, based on specified rules.

Each target group is associated with a protocol and port, and can be linked to an Application Load Balancer (ALB), Network Load Balancer (NLB), or Gateway Load Balancer.

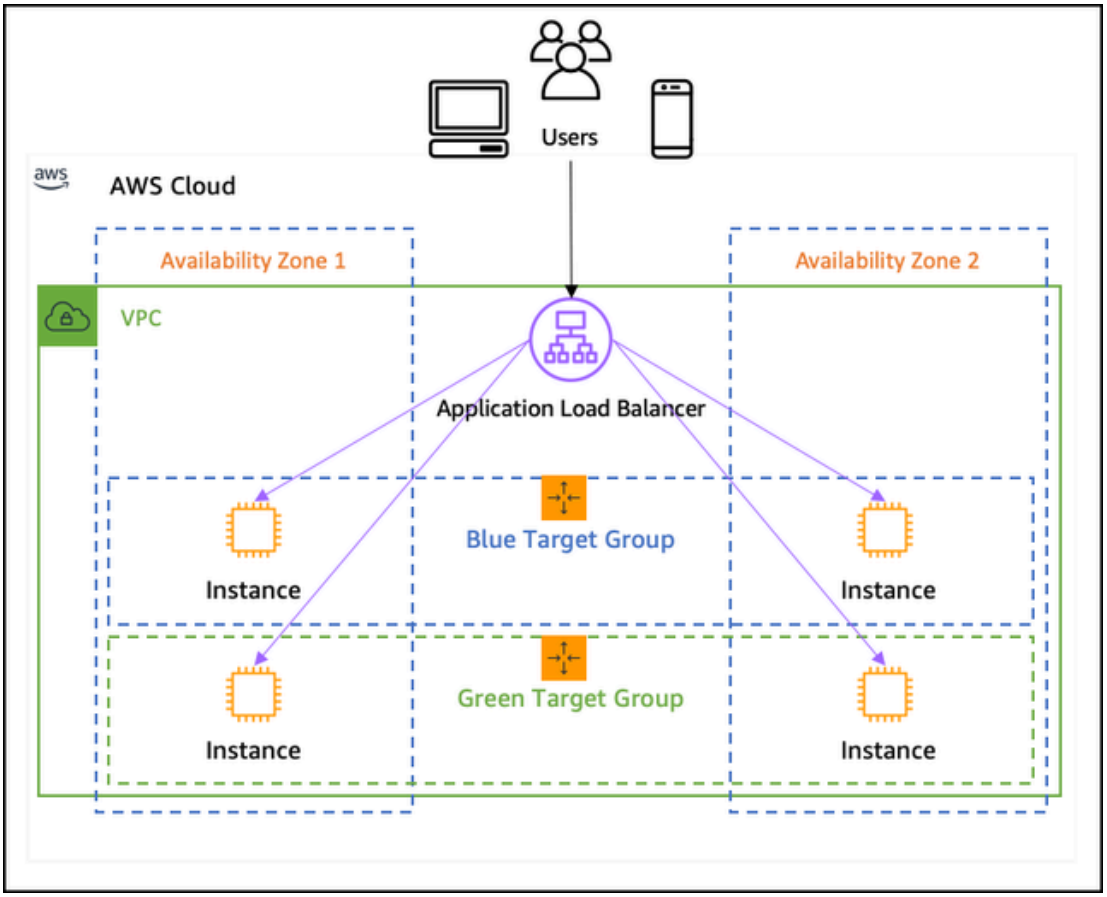
Target groups enable health checks, ensuring traffic only reaches healthy resources, and allow fine-grained traffic routing for microservices or specific application paths.

We choose a target group for ELB because it is required when practicing hands-on. A target group is linked with the Application Load Balancer, which we also use.

Application Load Balancer(ALB):

An Application Load Balancer (ALB) works at Layer 7 of the OSI model, routing HTTP/HTTPS traffic based on rules you define. It evaluates listener rules in order, then sends requests to the right target group. You can route traffic by URL, hostname, or other content, and even use different routing algorithms like round robin or least outstanding requests.

Targets can be added or removed without downtime, and ALB automatically scales with traffic. Health checks ensure requests go only to healthy targets, keeping your application available and responsive.



Hands-on practice:

Before diving into the hands-on practice, you need some prerequisites. You'll need at least two EC2 instances. For creating them, you can either create a new security group or use an existing one. For better results, configure different servers on each EC2 instance, for example, run Nginx on one and Apache on the other. This will help you clearly see which server is handling the requests.

Here the user-data script for configure server:

Apache:

```
#!bin/bash
apt update -y
apt install apache2 -y
systemctl start apache2
systemctl enable apache2
echo "<h1>Apache Server. Hostname is $(hostname -f)
</h1>" > /var/www/html/index.html
```

Nginx:

```
#!/bin/bash
apt update -y
apt install -y nginx
systemctl enable nginx
systemctl start nginx
echo "<h1>Nginx Server. Hostname is $(hostname -f)</h1>"
> /var/www/html/index.html
```

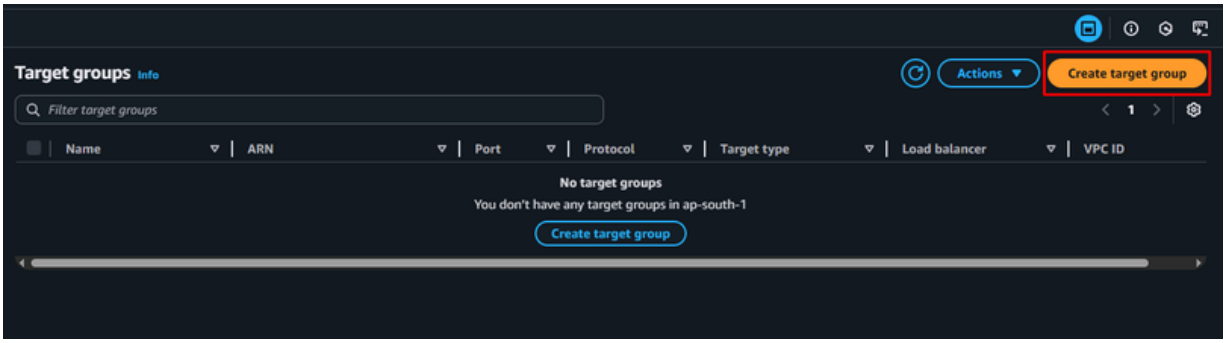
Hands-on practice:

To configure a Load Balancer, we first need to create a Target Group (TG). After configuring it successfully, we can proceed to create the Load Balancer using this TG. To create a TG, you can search for ‘Target Group’ or access it directly from your EC2 dashboard. Here are the instructions:

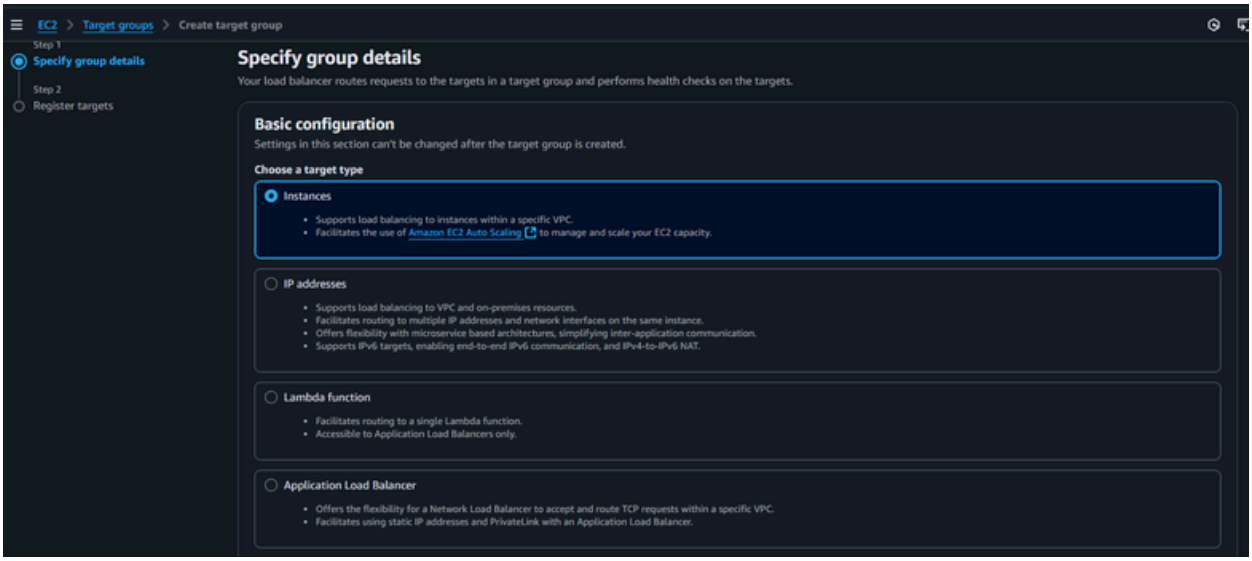


Hands-on practice:

Now, click on Target Groups, get this page. I didn't create any TG before that's why it shows empty.



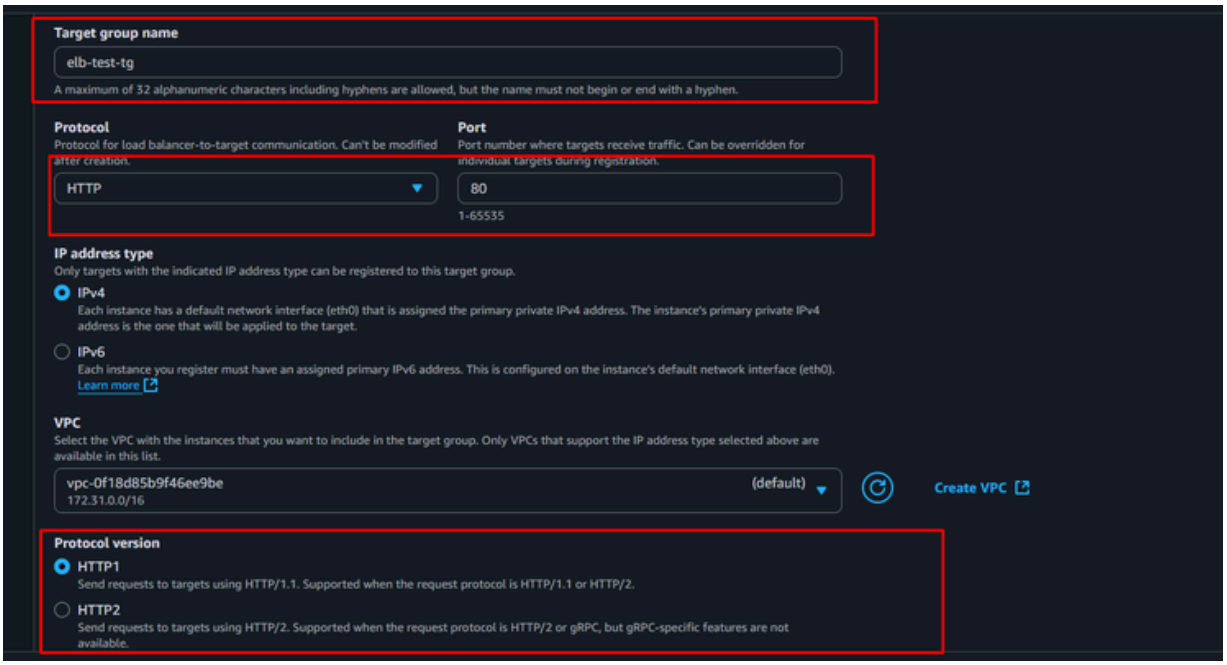
Click on Create Target Group button and get this configure page.



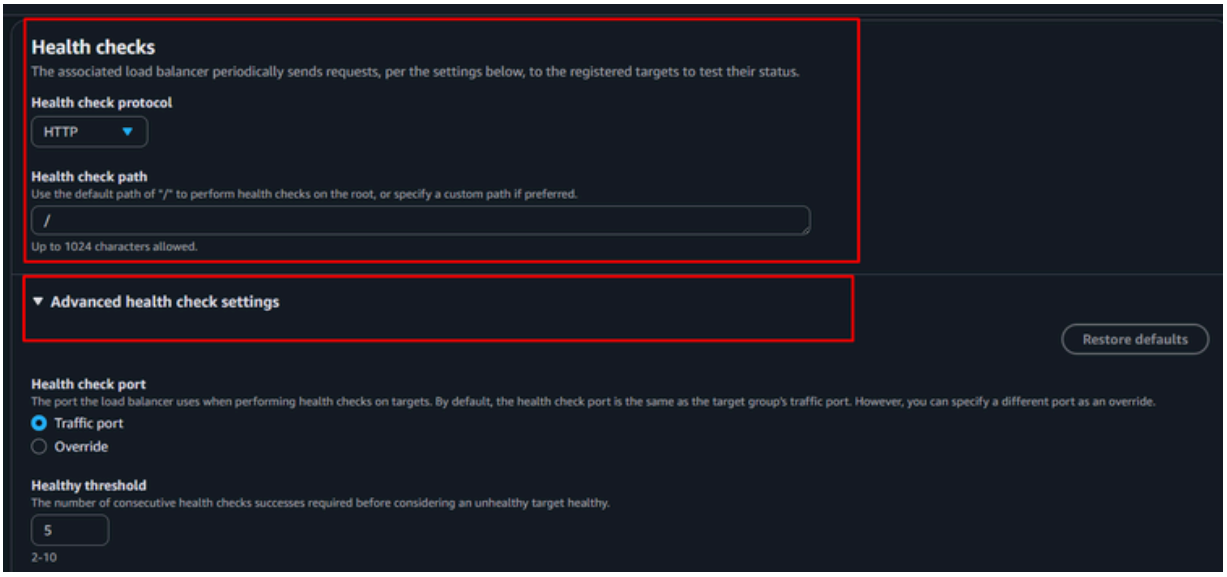
In my case, i choose an instance because i make TG with EC2 instace.

Hands-on practice:

Now fillup all the configuration based on your needs.

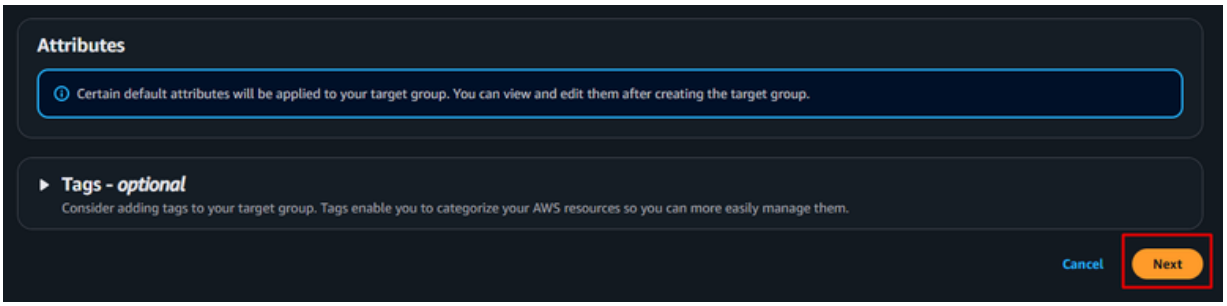


If you need advanced configuration for health checking, you can use. I kept all the things as it is.

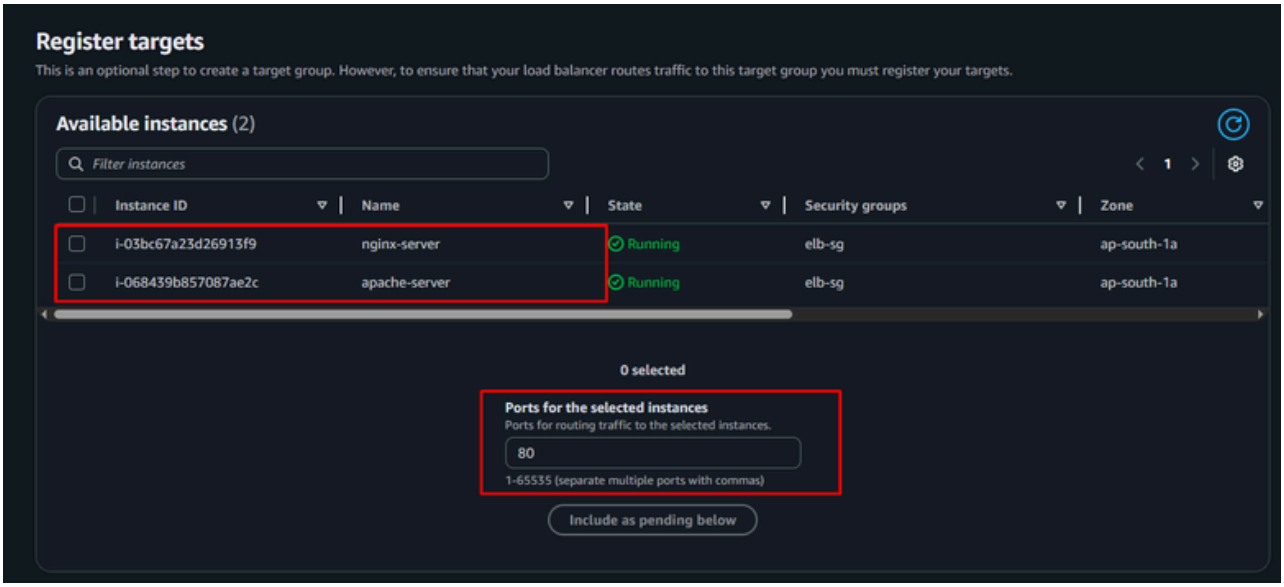


Hands-on practice:

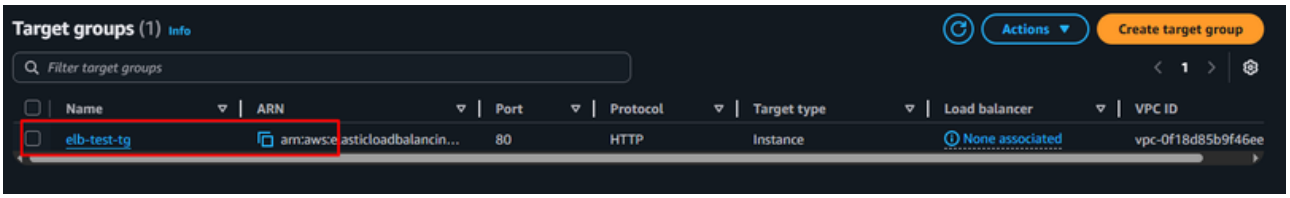
After complete the first step click on next button get the second step.

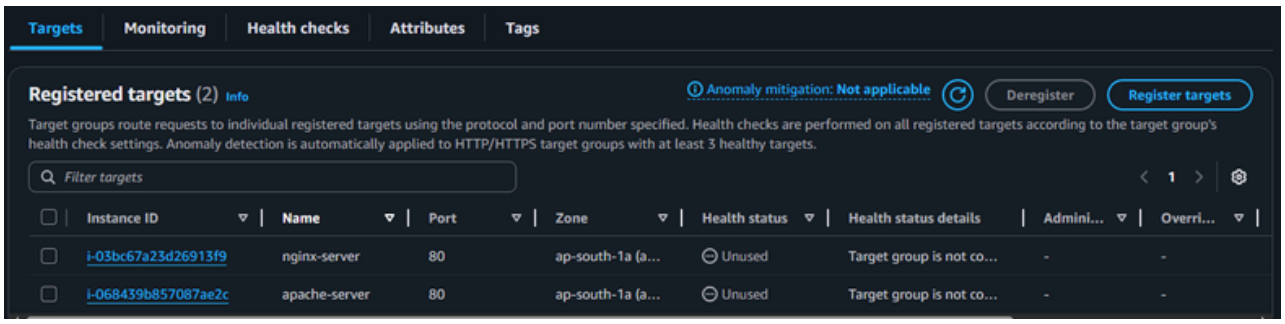


Now, register TG with select instances and set the port. And scroll down and click on create target group.



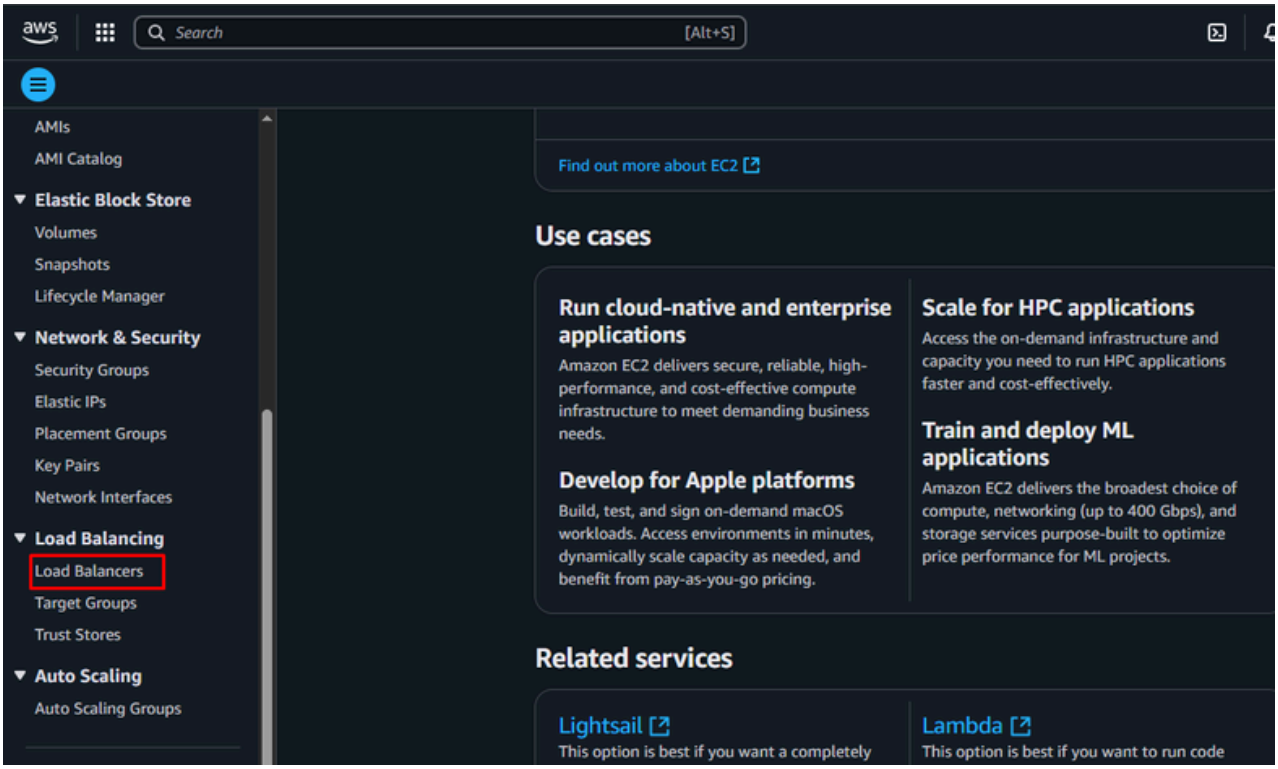
Successfully created TG, here it is.



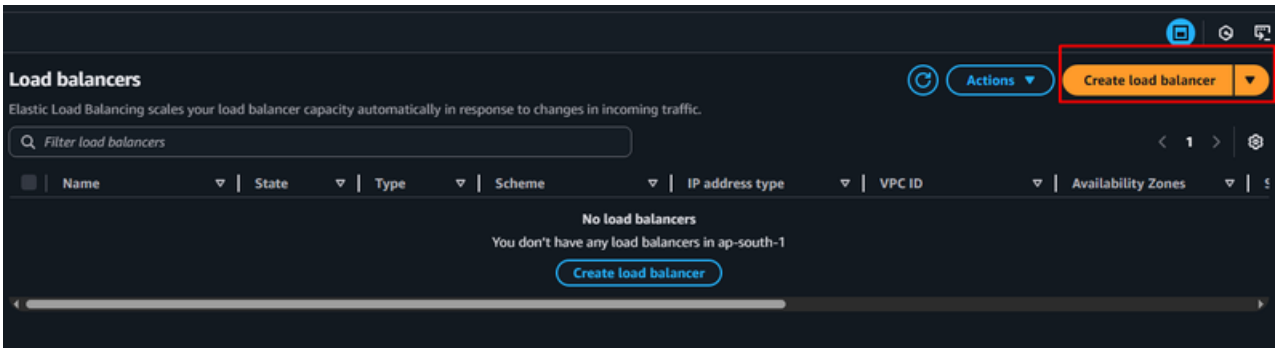


Hands-on practice:

Now it’s time to create Load balancer, for creating LB click on load balancers.

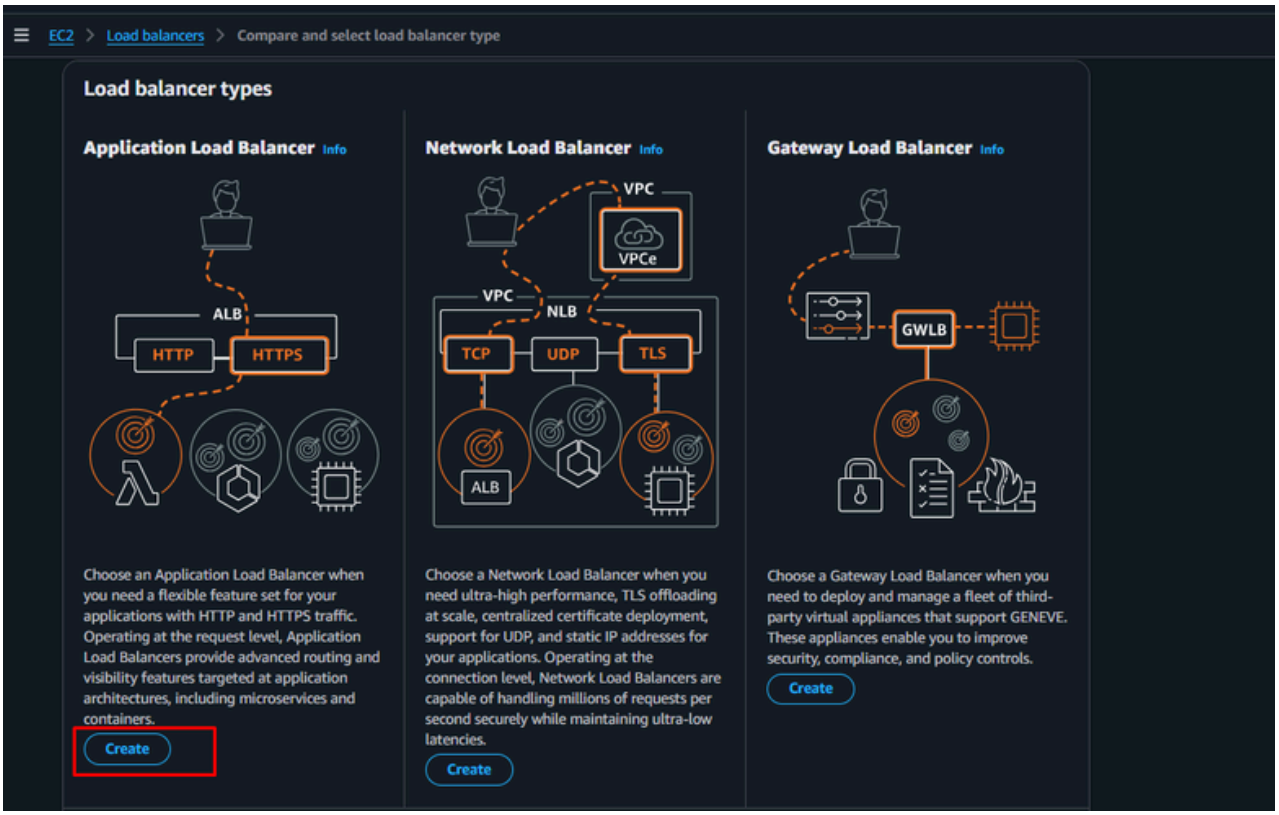


After clicking on Load balancer, you get this page. Now click on Create Load Balancer button.

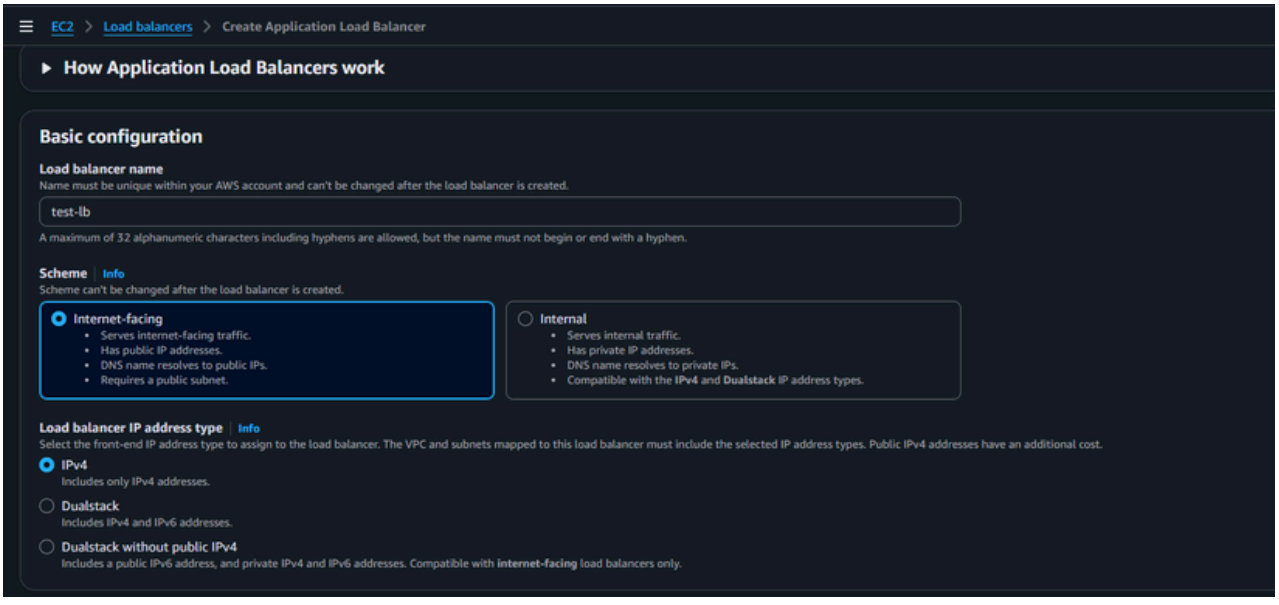


Hands-on practice:

Choose the LB, I will use ALB that I discussed before.

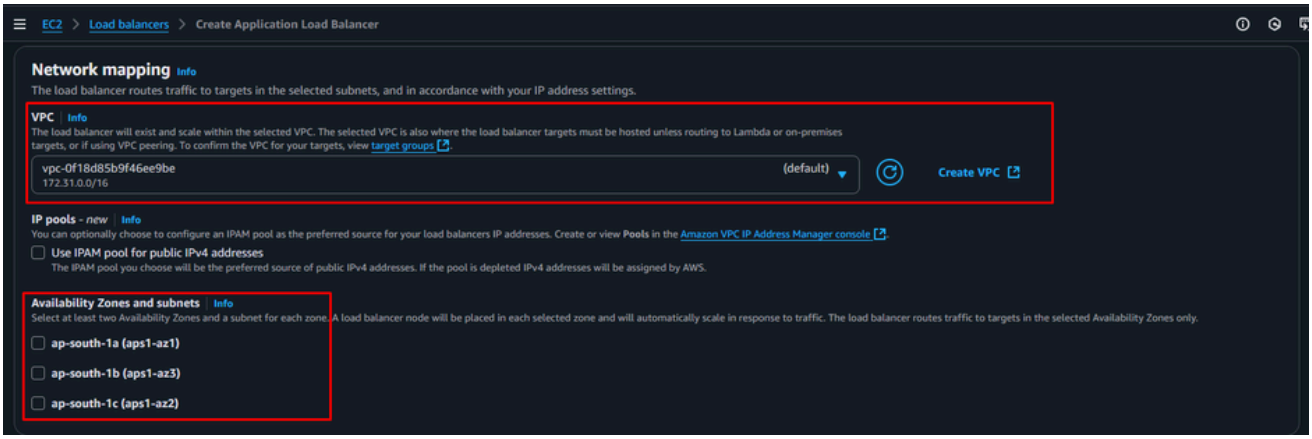


Now configure it based on you needs.

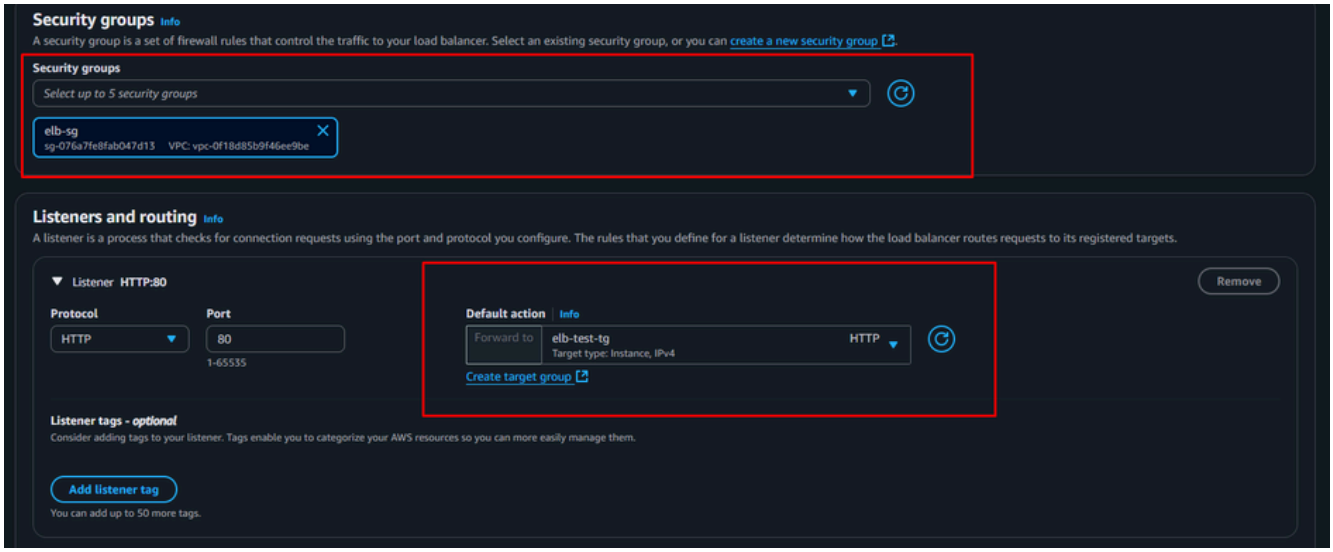


Hands-on practice:

In network mapping section, you can specify your custom VPC if you have any, then select the AZ.

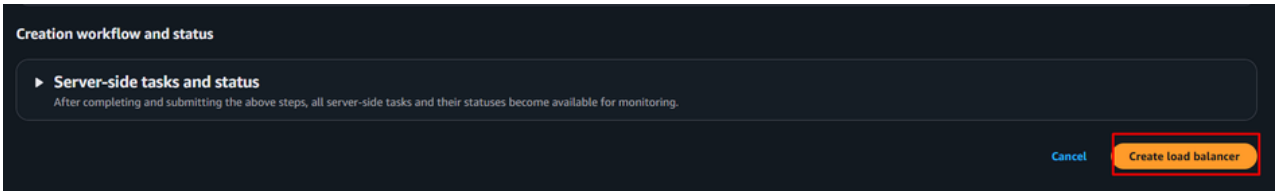


In the Security Group section, you can either add a custom SG or use the default one. In the Listeners and Routing section, select the TG you created earlier.

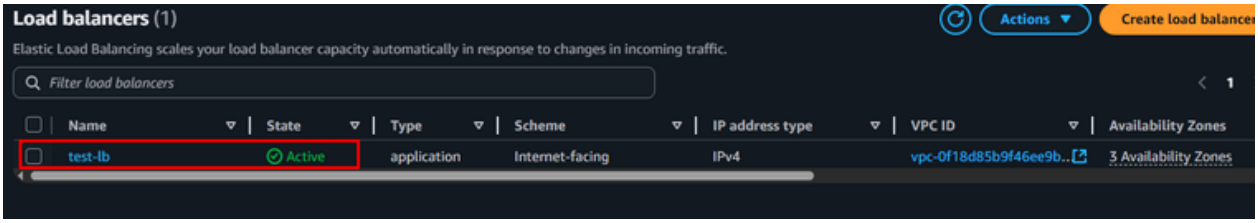


Hands-on practice:

If you need other configurations, you can configure them otherwise, scroll down and click on create load balancer button for create LB.



It takes some time to get ready, so please be patient and wait.
Now it's ready to serve.



Here the DNS name just copy it and browse from your browser.



The Load Balancer has been successfully created. Here are the results:



Apache Server. Hostname is ip-172-31-32-110.ap-south-1.compute.internal



Nginx Server. Hostname is ip-172-31-47-12.ap-south-1.compute.internal

Thank You

Stay Connect:

/in/alamgirweb11

/alamgirweb11